

# Programming in Modern C++: Assignment Week 2

Total Marks : 25

Partha Pratim Das  
Department of Computer Science and Engineering  
Indian Institute of Technology Kharagpur, Kharagpur – 721302  
partha.p.das@gmail.com

July 26, 2025

## Question 1

Consider the following program.

*[MCQ, Marks 2]*

```
#include <iostream>
using namespace std;

int main(){
    const int n = 20;  // LINE-1
    n = 30;            // LINE-2
    cout << n;
    return 0;
}
```

What will be the output/error?

- a) 20
- b) 30
- c) Compilation Error at LINE-1: `const` must be initialized
- d) Compilation Error at LINE-2: assignment of read only variable 'n'

**Answer:** d)

### Explanation:

The variable `n` is declared as `const` and correctly initialized to 20 at LINE-1, so there is no error there. However, at LINE-2, we attempt to modify a constant variable, which is not allowed. This causes a compilation error: `assignment of read only variable 'n'`.

## Question 2

Consider the following program.

*[MCQ, Marks 2]*

```
#include <iostream>
using namespace std;

int main(){
    int n = 10;
    int * const p = &n;
    int m = 20;
    p = &m;    // LINE-1
    cout << *p;
    return 0;
}
```

What will be the output/error?

- a) 10
- b) 20
- c) Compilation Error at LINE-1: assignment of read only variable 'p'
- d) Segmentation fault

**Answer:** c)

### **Explanation:**

Here, **p** is declared as a constant pointer to an integer. This means the address that **p** holds cannot change. At LINE-1, we try to reassign **p** to another address, which violates this rule. Hence, a compilation error occurs: **assignment of read only variable 'p'**.

### Question 3

Consider the following code segment and find the output.

*[MCQ, Marks 2]*

```
#include <iostream>
#define SQUARE(x) (x * x)
using namespace std;

int main(){
    int a = 4;
    int b = SQUARE(a + 1);
    cout << b;
    return 0;
}
```

What will be the output/error?

- a) 25
- b) 9
- c) 13
- d) 20

**Answer:** b)

#### **Explanation:**

The macro `SQUARE(x)` is expanded as `(x * x)`. So, `SQUARE(a + 1)` becomes `(a + 1 * a + 1) = 4 + 1 * 4 + 1 = 4 + 4 + 1 = 9`. Parentheses should be used properly in macros to avoid such errors. Correct expansion should be `((x) * (x))` if you want to get the evaluation in the form `(4+1) * (4+1)`.

## Question 4

Consider the following code segment.

*[MCQ, Marks 2]*

```
#include <iostream>
using namespace std;

int main(){
    int x = 4, y = 6;
    _____ int &z = x + y;    // LINE-1
    cout << z;
    return 0;
}
```

Fill in the blank at LINE-1 such that the code compiles.

- a) `const`
- b) `static`
- c) `inline`
- d) `register`

**Answer:** a)

### **Explanation:**

Expression `x + y` results in a temporary value. In C++, only a `const` reference can bind to a temporary. Hence, to bind the temporary result of `x + y` to a reference, we must declare it as a constant reference.

## Question 5

Consider the following code segment.

[MCQ, Marks 2]

```
#include <iostream>
using namespace std;

void modify(int x1, _____, _____){ //Line-1
    x2 = x1 * 2;
    *x3 = x1 + 5;
}

int main(){
    int a = 2, b, c;
    modify(a, b, &c);
    cout << a << " " << b << " " << c;
    return 0;
}
```

Choose the correct option for parameter list at LINE-1 such that the output is 2 4 7.

- a) int &x2, int \*x3
- b) int \*x2, int \*x3
- c) int &x2, int &x3
- d) int \*x2, int &x3

**Answer:** a)

### **Explanation:**

To reflect changes in **b** and **c**, **x2** should be passed by reference (to modify **b**), and **x3** should be passed by address (to modify **c**). The function call confirms that **b** is passed as a variable (so use reference) and **c** as address (so use pointer).

## Question 6

Consider the following code segment.

[MCQ, Marks 2]

```
#include <iostream>
using namespace std;

int max(int a, int b){
    return (a > b) ? a : b;
}

float max(float a, float b){
    return (a > b) ? a : b;
}

int main(){
    cout << max(3, 4.5);    // LINE-1
    return 0;
}
```

What will be the output/error?

- a) 4.5
- b) 3
- c) Compilation Error at LINE-1: call to 'max' is ambiguous
- d) 4

**Answer:** d)

### **Explanation:**

The literal 4.5 is double by default. Our overloads are `int, int` and `float, float`. No function takes `double, double`. So, C++ needs to perform a conversion.

Between narrowing `double → int` and `double → float`, the compiler picks: `double → int` is considered better than `double → float` because `double → int` loses fractional part but is still "primitive to primitive" in this context. It chooses `int max(int, int)` with implicit narrowing of 4.5 to 4.

## Question 7

Consider the following code segment.

*[MSQ, Marks 2]*

```
#include <iostream>
using namespace std;

void fun(int a, int b = 0){ cout << a + b << endl; }
void fun(double a, double b = 1.0){ cout << a * b << endl; }

int main(){
    fun(2);      // LINE-1
    fun(2.0);    // LINE-2
    return 0;
}
```

Which line/s will give compilation error?

- a) LINE-1
- b) LINE-2
- c) Both LINE-1 and LINE-2
- d) No error

**Answer:** d)

### **Explanation:**

Here, both function calls are valid. `fun(2)` calls the integer version with default `b=0`, and `fun(2.0)` calls the double version with default `b=1.0`. Hence, no ambiguity or error occurs.

## Question 8

Consider the following code segment.

*[MCQ, Marks 2]*

```
#include <iostream>
using namespace std;

enum class Color { Red = 1, Green, Blue };

int main(){
    Color c = Color::Red;
    cout << static_cast<int>(c + 1);    // LINE-1
    return 0;
}
```

What will be the output/error?

- a) 2
- b) Compilation Error at LINE-1: invalid operands to binary +
- c) 1
- d) 3

**Answer:** b)

### **Explanation:**

Scoped enums like `enum class` do not allow implicit conversions or arithmetic operations. `c + 1` is invalid unless overloaded. So, the compiler reports an error: `invalid operands to binary +`.

## Question 9

Consider the following code segment.

*[MCQ, Marks 2]*

```
#include <iostream>
#include <stdlib.h>
using namespace std;

int main(){
    int *p = new int[5];    // LINE-1
    p[0] = 42;
    cout << *p;
    delete[] p;
    return 0;
}
```

What will be the output?

- a) 0
- b) 42
- c) Compilation Error
- d) Segmentation fault

**Answer:** b)

### **Explanation:**

Memory is allocated using `new int[5]`, and the first element is set to 42. Then, `*p` accesses the first element. Hence, output is 42.

## Programming Question 1

Consider the program below.

- Fill in the blank at LINE-1 and LINE-2 such that the function doubles the value only if it is odd, otherwise the value remains unchanged.

*Marks: 3*

```
#include <iostream>
using namespace std;

int process(_____ n){           //LINE-1
    return (_____);             //LINE-2
}

int main(){
    int t;
    cin >> t;
    cout << process(t);
    return 0;
}
```

### Public 1

Input: 5

Output: 10

### Public 2

Input: 6

Output: 6

### Answer:

LINE-1: `const int &n`

LINE-2: `(n % 2 != 0) ? 2 * n : n`

### Explanation:

To check if a number is odd and double it, we use the ternary expression. Const reference is used to accept expressions as input.

## Programming Question 2

Complete the macro to calculate the average of two numbers after adding one to each number.

- Fill in the blank at LINE-1 so the macro works correctly.

*Marks: 3*

```
#include <iostream>
using namespace std;

#define AVG(X, Y) _____ // LINE-1

int main(){
    int a, b;
    cin >> a >> b;
    cout << AVG(a + 1, b + 1);
    return 0;
}
```

### Public 1

Input: 2 4

Output: 4

### Public 1

Input: 3 -5

Output: 0

### Private

Input: 3 5

Output: 5

### Answer:

LINE-1: `((X) + (Y)) / 2`

### Explanation:

The macro must properly parenthesize both arguments to avoid operator precedence errors. The correct macro is: `((X) + (Y)) / 2`.

## Programming Question 3

Fill in the function header to overload the '+' operator for a custom class.

- Implement operator+ to add values of two objects.

*Marks: 3*

```
#include <iostream>
using namespace std;

class Box{
    int value;
public:
    Box(int v): value(v) {}
    _____ operator+ (const ___ &b){          // LINE-1
        return Box(value + b.value);
    }
    void display(){ cout << value; }
};

int main(){
    int n1, n2;
    cin >> n1 >> n2;
    Box b1(n1), b2(n2);
    Box b3 = b1 + b2;
    b3.display();
    return 0;
}
```

### Public 1

Input: 5 10

Output: 15

### Public 1

Input: 15 5

Output: 20

### Private

Input: 15 15

Output: 30

### Answer:

LINE-1: Box

### Explanation:

The overloaded operator returns a new Box object with the sum of the internal values.